

# AI Skin Intelligence

---

AI Skin Intelligence is a personalized skincare companion that helps users understand their skin, track routines, review recommendations, and share insights with consultants and dermatologists. The platform combines a web dashboard, authenticated user flows, and an AI-backed skin analysis engine.

## Overview

---

This project includes:

- A front-end experience with HTML/CSS/JavaScript screens for login, onboarding, dashboards, and routines
- A backend API for accounts, profiles, assessments, and health checks
- A Python AI engine for skin analysis and recommendation scoring
- Role-based portals for users, consultants, dermatologists, and administrators

## Project structure

---

```
AI Skin Intelligence/
├── index.html           # Landing page
├── login.html           # Sign-in and account registration
├── dashboard.html       # User dashboard shell
├── user_dashboard.html  # User experience dashboard
├── consultant_dashboard.html # Consultant view
├── dermatologist_dashboard.html # Dermatologist view
├── admin_dashboard.html # Admin operations dashboard
├── routine_manager.html # Routine planning and management
├── report_view.html     # Report detail view
├── reminders.html       # Reminder and follow-up tools
├── skin_profile.html    # Skin profile management
├── src/                 # Shared CSS and JS
├── backend/             # Express API server
├── python-engine/       # Python AI and model logic
├── setup-db.ps1         # Database bootstrap helper
├── skin_assessment.db   # Local SQLite database used by the AI engine
└── README.md           # Project documentation
```

## Key features

---

### For users

- Create an account or sign in with email/password
- Use Google sign-in via the backend authentication flow
- Maintain a personalized skincare profile

- Track daily skincare routines and reminders
- Review skin health scores and recommendations
- Share health reports with clinicians

### For consultants and dermatologists

- View client or patient information and progress signals
- Review recommendation history and skin metrics
- Monitor adherence and treatment progression
- Share reports and care guidance

### For administrators

- Track platform health and operational metrics
- Review support workflows and user growth signals
- Monitor the health of the system and service status

### AI and analysis capabilities

- Skin assessment scoring using a Python model pipeline
- Personalized routine recommendations based on profile and trends
- Health checks and report summarization for care team review

## Platform architecture

---

### Frontend

The front-end is a static web application built using HTML, CSS, and JavaScript. Pages are linked from the main landing flow and are designed for role-based dashboards.

### Backend API

The backend is built with Node.js and Express and lives under the `backend/` folder. It handles:

- user authentication
- registration and role assignment
- profile management
- API endpoints for dashboards and reports
- health checks and environment configuration

### Python AI engine

The Python engine under `python-engine/` handles skin assessments, recommendation processing, and scoring logic. It uses a SQLite database and model artifacts to generate personalized reports.

## Prerequisites

---

Before running the project locally, install:

- Node.js 18+
- npm
- Python 3.10+
- pip
- Git (optional, but commonly used)

## Quick start

---

### 1. Clone the repository

```
git clone <your-repository-url>
cd "AI Skin Intelligence"
```

### 2. Install backend dependencies

```
cd backend
npm install
```

### 3. Configure environment variables

Copy the example environment file if present and update values as needed:

```
cp .env.example .env
```

Check the `backend/.env.example` file for required variables such as:

- database connection values
- JWT secret or auth configuration
- Google OAuth credentials
- application base URL or API host

### 4. Start the backend API

```
cd backend
npm start
```

Alternative in dev mode:

```
npm run dev
```

### 5. Start the frontend

Because the project uses static HTML pages, you can open the root pages directly in a browser or serve them through a local web server.

Example using Python:

```
cd "AI Skin Intelligence"  
python -m http.server 8000
```

Then open:

```
http://localhost:8000
```

## 6. Start the Python AI engine

From the project root:

```
cd python-engine  
pip install -r requirements.txt  
python start.py
```

If the AI engine is designed to run as a supporting service or research pipeline, follow the scripts and app entry points inside `python-engine/` for the exact workflow.

## Accessing the app

---

- Landing page: `index.html`
- Login/register: `login.html`
- User dashboard: `dashboard.html` or `user_dashboard.html`
- Consultant dashboard: `consultant_dashboard.html`
- Dermatologist dashboard: `dermatologist_dashboard.html`
- Admin dashboard: `admin_dashboard.html`

## User guide

---

### Create an account

1. Open the landing page.
2. Click `Start My Journey`.
3. Select the `Register` tab.
4. Enter your full name, email, password, and account type.
5. Complete registration.

Available account types include:

- User

- Consultant
- Dermatologist

Admin accounts are separate and typically use a specific admin sign-in flow.

## Sign in

1. Select the **Sign In** tab.
2. Enter your email and password.
3. Click **Sign In to Dashboard**.
4. If available, use the Google sign-in option for faster onboarding.

## Complete a skin assessment

Once logged in:

1. Open the profile or assessment flow.
2. Answer questions about your skin type, sensitivity, goals, and routine.
3. Save or submit the assessment.
4. Review your score and recommendation summary.

## Review dashboard recommendations

The dashboard provides a quick overview of:

- skin health score
- routine completion
- recommendations
- reminders
- report insights

Use the navigation buttons to move between Overview, Insights, Recommendations, Progress, and Reports.

## Manage your skincare routine

From the routine management area:

- review daily tasks
- add or update routine steps
- track morning and evening care tasks
- mark tasks complete as you go

## Share reports with your care team

1. Open the report or dashboard section.
2. Click the share option for reports.
3. Send the summary to your dermatologist or consultant.

4. Track whether the report has been viewed or used in care decisions.

## Consultant and dermatologist guide

---

### Consultant workflow

1. Sign in with a consultant account.
2. Open the consultant dashboard.
3. Review active client summaries and recommendations.
4. Check progress and interactions on the skin profile.
5. Share relevant updates and guidance.

### Dermatologist workflow

1. Sign in with a dermatologist account.
2. Open the clinical dashboard.
3. Review patient or user data trends.
4. Evaluate skin health summaries and recommendations.
5. Use final care decisions and communication tools to support treatment planning.

## Admin guide

---

1. Sign in using the designated admin flow.
2. Open the admin dashboard.
3. Review system health, user activity, and platform metrics.
4. Monitor operational health and respond to issues or service alerts.

## Troubleshooting

---

### API offline message

If the login page shows `API Offline`, confirm that the backend service is running.

Check:

```
cd backend
npm start
```

Then verify the health endpoint from your browser or terminal.

### Frontend pages do not load correctly

Make sure you are serving the project directory through a local web server or opening it from a browser that supports relative links.

```
cd "AI Skin Intelligence"
python -m http.server 8000
```

Then use:

```
http://localhost:8000
```

## Database or AI engine errors

Verify the dependencies are installed and the environment configuration is present:

```
cd backend
npm install

cd ../python-engine
pip install -r requirements.txt
```

If the issue involves the AI scoring engine, check the Python model files and SQLite DB under `python-engine/` and the project root.

## Security and privacy considerations

- Store JWT or session tokens securely.
- Use environment variables for credentials, secrets, and database settings.
- Keep OAuth credentials private and do not expose them in source code.
- Limit access to sensitive dermatology and skincare data to authorized users.
- Validate user input at the backend before storing or processing it.

## Recommended development flow

- Keep the frontend static pages and shared assets in the project root and `src/` directory.
- Manage API logic in `backend/`.
- Keep model logic and scoring files under `python-engine/`.
- Test database connections and authentication before deploying changes.
- Use a production-ready environment and update secret values before release.

## Production notes

The current login experience points to a hosted backend endpoint:

```
https://ai-skin.centralindia.cloudapp.azure.com/api
```

If you deploy to a different environment, update any hard-coded frontend API URLs and OAuth redirect configuration to match the new host.

## Support and next steps

---

For ongoing work, focus on the following priorities:

- improve user onboarding and skin assessment quality
- strengthen role-based authorization and privacy controls
- expand the AI scoring coverage and personalization logic
- tighten routine adherence tracking and reminder workflows
- review analytics and reporting quality for consultants and dermatologists

## Summary

---

AI Skin Intelligence is a skincare intelligence platform designed to help users better understand their skin, follow a consistent routine, and collaborate with care professionals. It blends a polished web experience with a secure backend and AI-assisted analysis to deliver personalized skincare guidance.